

Potato

- დროის ლიმიტი: 10 წუთი
- გარემო: ერთი GPU (≈16 GB VRAM), ინტერნეტის გარეშე
- ამოხსნის ზომა: `solution.ipynb` ≤ 1 MB
- მესხიერება: 5 GB

დავალება

შენი მეგობარი გთავაზობს გამოცნობას თამაშს. ის, როგორც მსაჯი, ირჩევს ერთ დაფარულ სიტყვას ფიქსირებული ლექსიკონიდან (vocabulary) და შენ ის მაქსიმუმ 30 სვლაში უნდა იპოვო. ყოველ სვლაზე მსაჯი ადარებს ორ სიტყვას და იტყობინება, რომელია სემანტიკურად უფრო ახლოს დაფარულ სიტყვასთან. ყოველი თამაში იწყება ფიქსირებული წყვილით `lamp` vs `potato`, რადგან ეს შენი მეგობრის ორი საყვარელი ნივთია. შემდეგ შენი პროგრამა გთავაზობს ერთ ახალ სიტყვას. შედარებაში გამარჯვებული სიტყვა რჩება და დარდება შენს შემდეგ შემოთავაზებულ სიტყვას. თამაშს იგებ იმ მომენტში, როდესაც ზუსტად შესთავაზებ დაფარულ სიტყვას. დამთხვევა ასოს სიდიდეზე მგრძნობიარე არ არის (case-insensitive). ყოველი სიტყვა, რომელსაც შესთავაზებ, უნდა იყოს `dataset/vocabulary.json`-ში.

სრული მაგალითი პროტოკოლითა (protocol) და მონაცემების ჩატვირთვით მოცემულია `solution.ipynb`-ში. შეგიძლია შეცვალო `PublicEmbeddingPlayer` კლასი. შენი პროგრამის ინიციალიზაცია ხდება ერთხელ და ის ყველა თამაშს ერთ გაშვებაში თამაშობს; პროტოკოლი ყოველი თამაშის დასაწყისში ქმნის ახალ `PublicEmbeddingPlayer`-ს.

მსაჯი

შენი პროგრამა აგზავნის ერთ JSON ობიექტს მსაჯთან და მსაჯი პასუხობს ერთი JSON ობიექტით.

დამუშავებული მაგალითი, სადაც დაფარული სიტყვა ნაჩვენებია მხოლოდ პროტოკოლის ასახსნელად:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}
-> {"status": "win"}

<- matches: game won
```

სვლები დაინდექსებულია 1-დან 30-მდე.

`verdict`-ის ოპციებია: `first`, რაც ნიშნავს, რომ `word1` უფრო ახლოსაა, `second`, რაც ნიშნავს, რომ `word2` უფრო ახლოსაა, ან `same`, რაც ნიშნავს, რომ ორივე სიტყვა თანაბრად ახლოსაა დაფარულ სიტყვასთან.

`winner_word` არის სიტყვა, რომელიც ინარჩუნებს ადგილს შემდეგი შედარებისთვის. `same` ვერდიქტისას რჩება პირველი სიტყვა.

მონაცემთა სიმრავლე

საერთოა თითოეული დაყოფისთვის (split):

- `dataset/vocabulary.json` — 1602 უნიკალური ქვედა რეგისტრის სიტყვა. დაფარული სიტყვა ყოველთვის ერთ-ერთი მათგანია.
- `dataset/public_embeddings.npy` — `float32`, განზომილება (shape) (1602, 2560). სტრიქონი `i` შეესაბამება ლექსიკონში არსებულ სიტყვას `i`. ეს არის საჯარო ემბედიგები (public embeddings); მსაჯი იყენებს განსხვავებულ, კერძო (private) წარმოდგენას.

დაყოფები (splits) დაფარული სიტყვების სიმრავლეებია:

დაყოფა (Split)	სიტყვები	პასუხები	გამოიყენე ამისთვის
<code>test_public</code>	120	<code>dataset/test_public.json</code>	შენი ამოხსნის გასაშვებად და საკუთარი ქულის შესაფასებლად
<code>test_leaderboard_a</code>	120	დაფარულია	ცოცხალი ლიდერბორდისთვის (live leaderboard)
<code>test_leaderboard_b</code>	120	დაფარულია	საბოლოო რანჟირებისთვის

`train` დაყოფა არ არსებობს — დაღლებული სტრიქონებიდან არაფერია დაფიქტული (*fitted*).

მონოდებული მოდელები

დავალებას მოჰყვება ორი წინასწარ დატრენინგებული ემბედინგის მოდელი, რომელთა გამოყენებაც შესაძლებელია:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/iaai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/iaai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

ორივე უნდა ჩაიტვირთოს მათი ლოკალური მისამართიდან (*path*); Hugging Face hub ID, როგორცაა "BAAI/bge-m3", იწვევს ჩამოტვირთვას და იშლება შეცდომით, რადგან შეფასება ხდება ოფლაინ რეჟიმში. თითოეული დირექტორია შეიცავს გაშვებად `example.py`-ს, რომელიც აჩვენებს ოფლაინ გამოძახებას.

ხელმისაწვდომი ბიბლიოთეკებია: `numpy`, `torch`, `sentence-transformers`. ინტერნეტის გარეშე, ჩამოტვირთვების გარეშე, სხვა პაკეტების გარეშე.

გამოსატანი მონაცემები

არაფერი. ეს არის ინტერაქტიული დავალება: შენი ამოხსნა არ წერს არანაირ პასუხების ფაილს; ის ესაუბრება მსაჯს `stdin/stdout`-ის საშუალებით, როგორც ზემოთ არის აღწერილი.

მეტრიკა

თამაში, რომელიც იპოვის პასუხს $\pm$ სვლაზე, იღებს ქულას $1.0 - 0.02 \times \max(0, t - 10)$; თამაში, რომელიც ვერ ამოიხსნება 30 სვლის განმავლობაში, იღებს ქულას 0. ასე რომ, 1–10 სვლები იღებენ ქულას 1.00, მე-20 სვლა იღებს ქულას 0.80, მე-30 სვლა იღებს ქულას 0.60.

შენი დავალების ქულა არის თამაშის საშუალო ქულა $\times 100$, 0.00-სა და 100.00-ს შორის.

10-ნუთიანი ლიმიტი არის ერთიანი ბიუჯეტი, რომელიც ფარავს გაშვებას, მომზადებასა და სატესტო სიმრავლის (*test set*) ყველა 120 თამაშს.

როგორ წარვადგინოთ ამოხსნა

- გახსენი `solution.ipynb`, დაარედაქტირე `PublicEmbeddingPlayer` და გააშვი ყველა უჯრა (*cell*), რათა დარწმუნდე, რომ ის მუშაობს.
- სურვილისამებრ, შეამოწმე ის ლოკალურად: `python local_test.py solution.ipynb --limit 5`. ლოკალური მსაჯი იყენებს *საჯარო* ემბედინგებს, ამიტომ მისი ქულა მხოლოდ ორიენტირია.
- შეინახე `solution.ipynb`.
- გახსენი Git ტაბი JupyterLab-ის მარცხენა გვერდითა პანელში.
- დაასტეივე (*Stage*) `solution.ipynb` (მის გვერდით არსებული + იკონკა).
- შეიყვანე ქომათის შეტყობინება (*commit message*) და დაანკაპუნე *Commit*-ს.
- დაანკაპუნე ღრუბლისა და ზემოთ მიმართული ისრის იკონკას *Push*-ის გასაკეთებლად.
- დაბრუნდი კონკურსის (*Contest*) ამ გვერდზე და დაანკაპუნე *Submit*-ს, სადაც ქომათის შეტყობინება ემოხვევა შენ მიერ მითითებულს.

წარადგინე ზუსტად ერთი ფაილი, სახელად `solution.ipynb`, რომელიც მოიცავს ყველა საჭირო მომზადებასა და ინფერენსს (*inference*).